Applied Software Development Lab
Lab Handout 1: Blinking LED

Learning objectives

- **Working in a Unix terminal:** using basic commands such as `cat`, `ls`, `cd`, `pwd`, `mkdir`; understanding how the Unix file system is organized hierarchically and how to move around in it; creating and editing simple text files with an editor such as `nano`.
- **Programming the microcontroller directly:** writing a simple program that makes an LED on a specific pin (e.g., pin 2) blink – at first directly in machine code, to get a feeling for how the microcontroller "understands" programs.
- **Measuring with the oscilloscope:** understanding why the LED no longer visibly blinks when the program is very fast, and how an oscilloscope can be used to check the actual behavior of the program.
- **Understanding the toolchain for microcontroller programming:** seeing which tools are used to transfer a program to the microcontroller – e.g., how `avr-as`, `avr-ld`, and `avr-objcopy` turn assembly code into machine code, and how `avrdude` is used to flash it to the microcontroller (e.g., an Adafruit Metro with an ATmega328P). The goal is to understand what happens "behind the scenes" when you later press a button in the Arduino IDE.

How to use this lab handout

This lab handout is primarily a practical guide for carrying out individual tests and experiments. It shows step by step what to do and, where useful, contains brief notes on the meaning of individual commands or steps. The technical background is deliberately only touched upon.

A more detailed explanation of the theoretical and technical background can be found in the separately provided **project companion**. The two documents can be used independently of each other: for example, you can first work through the lab handout to get a feeling for the procedures and read up on the background afterwards – or the other way round. Switching back and forth between the two approaches works as well. Everyone can choose their own way in – depending on learning style and interest.

Step 1: Our first program for the ATmega328P

In this step, first create a file `led_pin2.hex` with the following content:

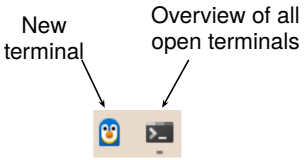
```
:08000000529A5A9A5A98FDCF5A
:00000001FF
```

This file contains machine code for the ATmega328P. A more detailed explanation can be found in the **project companion**; a brief explanation follows further below in this handout.

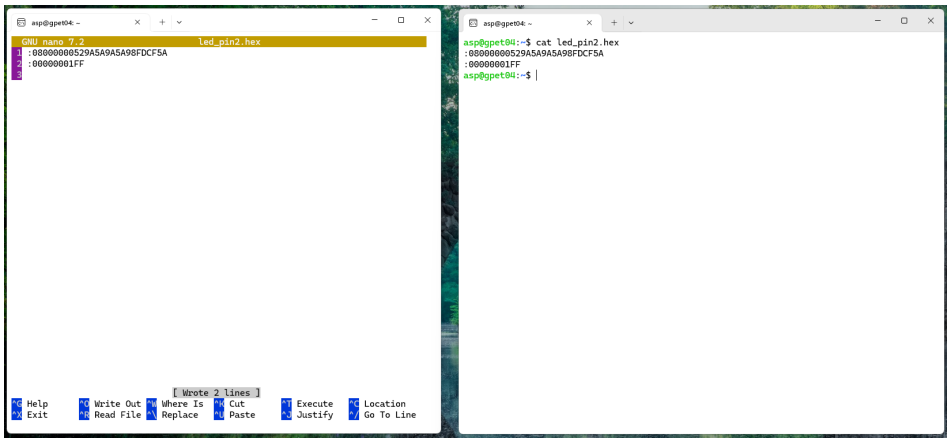
The program `avrdude` can write this machine code into the program memory of the microcontroller (this is also called "flashing").

Two terminals – why?
For this part of the work, it is a good idea to use two terminal windows. In the lab we use Windows PCs, but for our work we use Linux via WSL (Windows Subsystem for Linux).

In the taskbar at the bottom of the screen you will find the icon for opening a new Linux terminal and the overview of all terminals that are already open.



The best way is to open two terminals side by side – for example like this:



Left terminal	Right terminal
Editor (nano)	Tools such as `ls`, `cat`, `avrdude`

The advantage: a stress-free workflow. You edit your file on the left and test it or enter commands on the right. If, for example, an error occurs, tools often report a line number – and you have it right in front of you when you make the correction on the left.

Tasks:

1. **Create the file and enter its content**
   (a) In the left terminal, enter the following command:

   ```
   nano led_pin2.hex
   ```

   This starts the text editor `nano` with a (still empty) file named `led_pin2.hex`.
   (b) Now enter the code shown above (two lines).
   Make sure to copy it exactly – including the colons. The letters A–F may be entered in lowercase or uppercase here, but keep in mind that in many programming languages the case of letters does make a difference. By the way, a not uncommon mistake is typing `O` (“oh”) instead of `0` (“zero”)!
   (c) Save the file with the key combination `Control` + `S`.

2. **First steps with Unix commands**
   In the right terminal you can now check the file:
   (a) Enter:

   ```
   ls
   ```

   This lists all files in the current directory. The file `led_pin2.hex` should now appear.
   (b) Enter:

   ```
   cat led_pin2.hex
   ```

   This prints the content of the file in the terminal. Compare it with what you entered in nano on the left.

© Michael Lehn | CC BY-SA 4.0

Step 2: Connecting the microcontroller and detecting the device

Before we connect the microcontroller, let us briefly look at how you can tell under Linux whether a device (e.g., a USB-to-serial converter) has been recognized by the system.

Terminal tip:
If you want to enter a command again in the terminal, you can simply press the up-arrow key. This is especially handy when you run a command repeatedly or want to modify it slightly.

Tasks:

1. **Do not connect the microcontroller yet**
   Enter the following command in the terminal:

   ```
   ls /dev/*
   ```

   This command lists all device files located in the directory `/dev/`. Under Linux, almost everything is treated as a "device file" – including USB devices. A USB-to-serial converter like the one on our microcontroller board would show up here, e.g., as `/dev/ttyUSB0`.

   Since no microcontroller is connected yet, the output should of course **not** contain an entry like `/dev/ttyUSB0`.

2. **Connect the microcontroller**
   Now connect the microcontroller to the computer with the USB cable. On a real (native) Linux system, running

   ```
   ls /dev/*
   ```

   again would now show an additional entry such as `/dev/ttyUSB0` – that would be your device.

   Since we are working in WSL under Windows, however, Linux runs as a "guest" here, and the USB port has to be passed through. For this purpose, we have prepared a helper script called `usb_check`.

3. **Run `usb_check`**
   Enter:

   ```
   usb_check
   ```

   The script gives WSL access to the USB port. Afterwards, check again with:

   ```
   ls /dev/*
   ```

   whether `/dev/ttyUSB0` now appears. If so: everything is ready for flashing!

What to do if there are problems?
If `usb_check` reports an error or asks for the administrator password: don't panic. Such things can happen – especially when workarounds were still being tinkered with shortly before the start of the semester 😄 Sit back for a moment and stay relaxed – we will help you shortly.

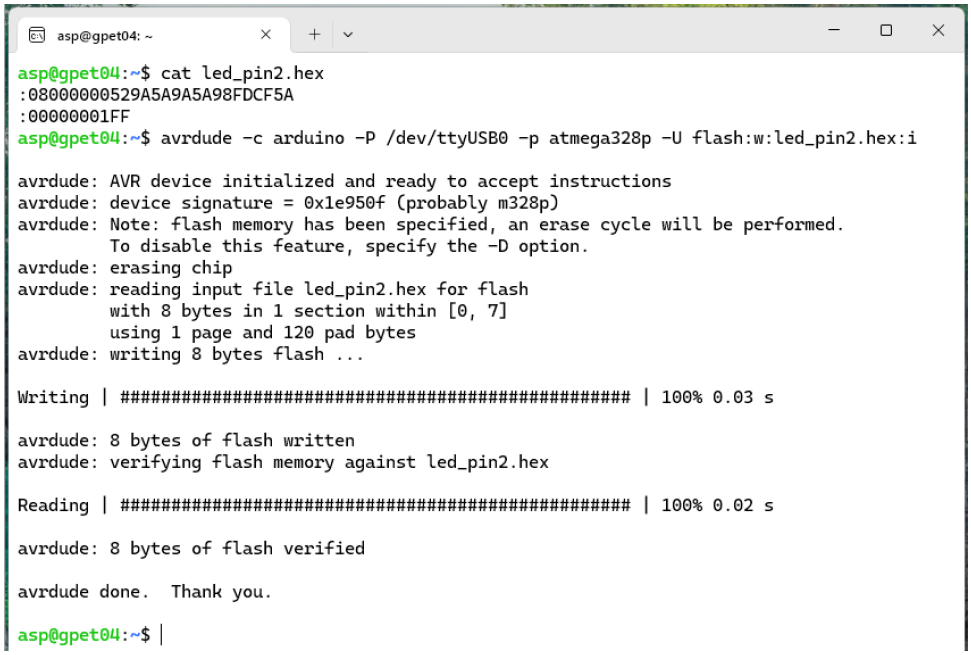
© Michael Lehn | CC BY-SA 4.0

Step 3: Flashing the microcontroller

Now it is getting serious: we transfer our program to the microcontroller – that is, into its flash memory, where it is executed automatically after power-up.
To do this, enter the following command in the right terminal:

```
avrdude -c arduino -P /dev/ttyUSB0 -p atmega328p -U flash:w:led_pin2.hex:i
```

If everything works, the output of `avrdude` looks roughly like this:



Now we make a small case distinction – just like when programming with `if` and `else`:

- If the output contains something like:

  ```
  8 bytes of flash written
  8 bytes of flash verified
  ```

  then flashing was successful. Continue with the next step!

- If, on the other hand, `avrdude` reports an error – or “verifying failed” appears – then:

  Back to Step 1! Most likely a typo crept in while you were entering the hex file. Go back to `nano`, check everything, save again, and then try once more.

Errors are part of the workflow
Errors during flashing (or during programming in general) are completely normal. What matters is that you understand them **as part of the workflow**:

- You recognize that an error has occurred (e.g., from unusual `avrdude` output),
- you learn not to overlook such messages, but eventually to notice them even subconsciously – the way you notice a red traffic light without consciously looking at it,
- and you know which step you have to go back to in order to fix it.

This is exactly what working professionally means: errors are not a step backwards – they are the way to the solution.

© Michael Lehn | CC BY-SA 4.0

Step 4: Measuring what the program does

Congratulations – you have made it to Step 4!
The machine program has already been transferred successfully to the microcontroller and is running now. So it is about time to find out what we have actually done there.

You can find the details in the project companion – but, as everyone knows, people like to read a manual only afterwards. So here is the most important information in brief. Our program consists of exactly four machine instructions:

- set pin 2 as an output
- set pin 2 HIGH
- set pin 2 LOW
- and a jump instruction

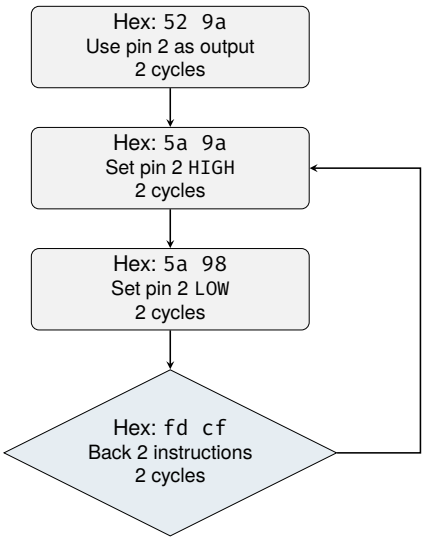


Fig. 1: Flowchart

These instructions are executed one after the other, as shown in Figure 1. Each of these instructions takes two CPU clock cycles. At a clock frequency of 16 MHz, one clock cycle lasts exactly 62.5 ns. This means: pin 2 should be HIGH for 125 ns, then LOW for 250 ns – and then the sequence starts all over again. Figure 2 sketches this sequence.

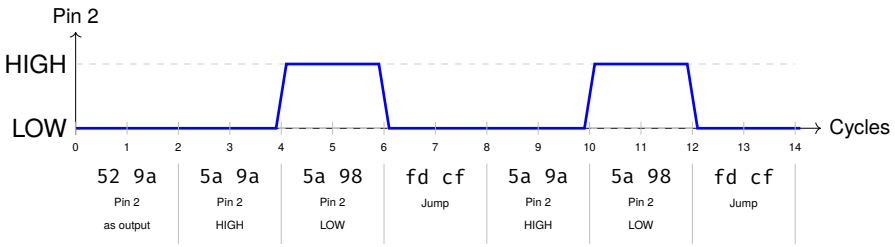
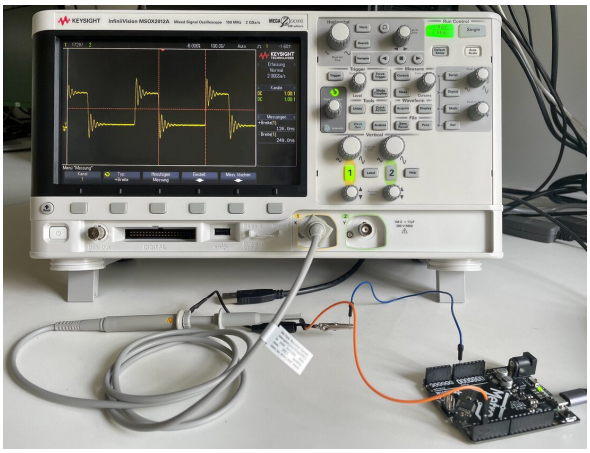
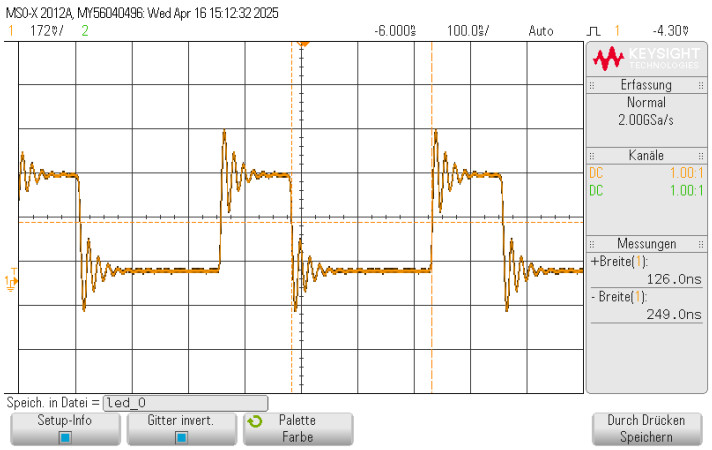


Fig. 2: Signal at pin 2.

But anyone can claim that. We, however, measure it – with the oscilloscope!





1. Connect the probe:
   Channel 1 (yellow) to pin 2 of the microcontroller, ground lead (cable with alligator clip) to `GND`.
2. Set the trigger:
   Mode: `Edge`, Source: `Chan 1`, Level approx. 2.5 V
3. Adjust the time base:
   `Time/Div`: approx. 100 ns
4. Enable automatic measurement:
   Press the `Meas` key and select one after the other:
   `+Width` (measures the HIGH duration),
   `-Width` (measures the LOW duration)

© Michael Lehn | CC BY-SA 4.0

Step 5: A small hack in the machine code

The project companion explains the so-called Intel HEX format in detail. Here is the short version. The file `led_pin2.hex` contains more than just the actual program. In addition to the machine program, it contains, for example:
- how many bytes are to be transferred
- the address to which they are to be written
- a checksum for error detection

The first line, for example, has the following structure:

```
:08 0000 00 529A5A9A5A98FDCF 5A
08 = number of data bytes, 0000 = start address, 00 = record type (data), 5A = checksum
```

The machine program itself consists only of the bytes highlighted in green:

```
529a5a9a5a98fdcf
```

By the way, the second line of the file (`:00000001FF`) simply means: no more data follows.

Now here comes the clever part: The checksum in the Intel HEX format detects many kinds of transmission errors – but not all of them. If two consecutive instructions are swapped, the behavior of the program changes – but the checksum stays the same.

Task:
In `led_pin2.hex`, swap the instructions for “set pin 2 HIGH” and “set pin 2 LOW”. If everything works, you should see on the oscilloscope: 250 ns HIGH and 125 ns LOW.

Difference from Step 4: There, we measured in order to understand what happens. Now we make a prediction and then check whether it is correct.

Step 6: A hack for a symmetric signal

The project companion explains how to modify the program so that pin 2 is **HIGH for 250 ns** and then **LOW for 250 ns**.

The signal then consists of phases of **equal length** – it is therefore **symmetric** and has a clean frequency.

To achieve this, we simply insert two additional instructions, so-called **NOPs**:
- NOP stands for “No Operation” – that is, “do nothing”.
- The instruction takes one CPU clock cycle but does not change the state of the pin:
  - If pin 2 is HIGH, it stays HIGH.
  - If it is LOW, it stays LOW.

In our case, we insert the two NOPs directly after the “set pin HIGH” instruction. This extends the HIGH phase by

125 ns + 125 ns = 250 ns.

Figure 3 shows exactly what this looks like.

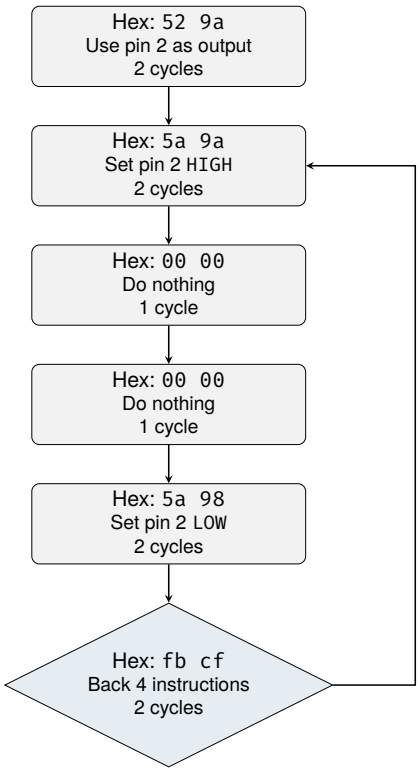


Fig. 3: Symmetric signal

© Michael Lehn | CC BY-SA 4.0

The new program now consists of the following bytes:

```
529A5A9A000000005A98FBCF
```

So, in summary:
In `led_pin2.hex`, only the first line needs to be changed.

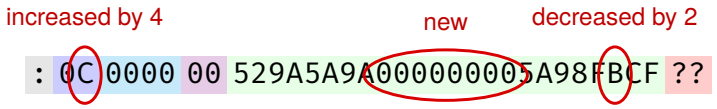
The only tricky part: what is the new checksum?

```
: 0C 0000 00 529A5A9A000000005A98FBCF ??
```

Of course, you could look up in the project companion exactly how it is calculated. But here, the essentials are enough:

> First, all bytes except the checksum are added up – always calculating with two hex digits, i.e., with 8-bit overflow: higher digits are simply cut off. The checksum is then exactly the number that would still have to be added so that the result is 0 again.

The crucial point for us: we only need to look at **where something has changed** and **by how much the sum has changed**.



And these are exactly three things:

1. The number of bytes has increased by 4.
2. The two new NOP instructions (`00 00 00 00`) contribute 0 to the sum.
3. The jump instruction has changed: `FD CF` becomes `FB CF`, i.e., the first byte is smaller by 2.

So the sum has become smaller by 2
→ the checksum must therefore become larger by 2 to compensate.
→ The old checksum was 5A, so the new one should be 58.

Task:

Modify `led_pin2.hex` by
- inserting the new machine instructions,
- increasing the data length accordingly,
- and changing the checksum from 5A to 58.

If we have made neither a reasoning error nor a calculation error, `avrdude` will accept the checksum without complaint, and the oscilloscope will show a symmetric signal.

© Michael Lehn | CC BY-SA 4.0

Intermezzo: A look at the end

In Step 7 we clone a finished game and play it together. Not because we want to analyze the code – but to get an impression of where this course is heading. What you are about to play is an example of what is possible at the end:

A self-written game in Python that can be controlled with a microcontroller.

What exactly you program is, of course, up to you. For classics such as Pong or Breakout (the game in which you bounce a ball off a paddle at the bottom and hit blocks at the top), there will be step-by-step instructions – but also plenty of room for your own ideas.

Over the course of the semester, we move from hardware-oriented programming (machine code, assembly language, ports, timing …) all the way to platform-independent software development in Python. The point is not to memorize solutions, but to understand which problem is being solved in each case – and why a particular approach makes sense. Once you know the problem, you also understand the purpose of the solution – whether it is about assembly language, C, or object-oriented programming. And you can better judge when and how to use such tools sensibly.

Step 7: Playing TRON Lightcycle

To round off the first session, we play a game!

At first glance, the game may remind you of Snake – but it has different roots: **TRON** was a science fiction film from 1982 and the first feature film to make extensive use of **computer-generated imagery (CGI)**. Its lightcycle races have inspired many game ideas – including this one.

Tasks

1. The game is available on GitHub at: https://github.com/michael-lehn/tron.git

   To clone it, enter the following in the terminal:

   ```
   git clone https://github.com/michael-lehn/tron.git
   ```

2. **Who does what?**
   The game can be played by four people – which fits perfectly, because the lab has four computers per row. One of you starts the server, the others connect to it with the client.

   (A) Starting the server: Open a terminal, change into the directory, and start the server:

   ```
   cd tron
   python new-tron-server.py 1000 1000 4
   ```

   This starts a game on a 1000 × 1000 grid for four players. Whoever starts the server should also look up their own IP address and tell it to the others:

   ```
   curl -4 ifconfig.me
   ```

   (B) Starting the client: The clients are started on each computer as follows:

   ```
   cd tron
   python tron-2d.py      (or python tron-3d.py)
   ```

   On the start screen, you can then enter your name and the IP address of the server.

© Michael Lehn | CC BY-SA 4.0